

System Design Prereqs: Deep Dive Fundamentals

Master the basics before scaling architecture

⇒ NETWORK FUNDAMENTALS ⇒

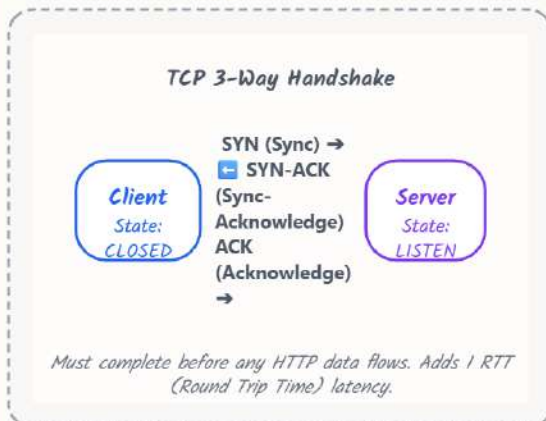
The Backbone of the Internet

★ 1. TCP/IP Architecture

The universal language of the internet. It dictates how data is packaged, addressed, and routed.

IP (Internet Protocol): The "addressing" system. Routes packets to the correct destination IP. Connectionless.

TCP (Transmission Control Protocol): The "delivery" system built on IP. Ensures reliable, ordered, error-checked delivery.



★ 2. DNS (Domain Name System)

Translates human-friendly domains (google.com) to machine IPs (142.250.190.46).

The Resolution Journey:

1. **Browser Cache:** Check local memory.
2. **OS Cache:** Check `/etc/hosts`.
3. **Recursive Resolver:** Usually your ISP.
4. **Root Nameserver:** Knows where TLDs are.
5. **TLD Nameserver:** (e.g., .com, .org).
6. **Authoritative Nameserver:** Holds the actual A Record (IP).

DNS Record Types	
A Record	Domain → IPv4
AAAA Record	Domain → IPv6
CNAME	Domain → Domain (Alias)

★ 3. The Request Lifecycle (What happens when you press Enter?)

- 1 **DNS Resolution:** Find the IP address.
- 2 **TCP Handshake:** Establish connection (SYN, SYN-ACK, ACK).
- 3 **TLS Negotiation (if HTTPS):** Exchange keys, establish secure tunnel.
- 4 **HTTP Request:** Client sends GET/POST request with headers.
- 5 **Server Response & Render:** App processes, DB query, sends HTML/JSON back. Browser renders.

Communication Protocols

★ 4. HTTP Anatomy (Stateless)

HTTP doesn't retain memory between requests. Every request must contain all info needed.

Anatomy of a Request

```
POST /api/users/login HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer token123

{
  "email": "user@test.com",
  "password": "hashed_pw"
}
```

★ 5. Core HTTP Methods

- **GET (Read):** Fetch data. *Idempotent (safe to repeat).*
- **POST (Create):** Submit new data. *Not Idempotent.*
- **PUT (Update/Replace):** Replace entire resource. *Idempotent.*
- **PATCH (Modify):** Partial update.
- **DELETE (Remove):** Delete resource.

★ 7. REST API Design Principles

Representational State Transfer. A standard architectural style for web APIs.

Resource-Based URLs (Nouns, not Verbs)

✗ Bad Design

```
GET /getAllUsers
POST /createNewUser
POST /deleteUser?id=5
```

✓ RESTful Design

```
GET /users
POST /users
DELETE /users/5
```

★ 8. JSON (JavaScript Object Notation)

The universal data format for modern APIs. Key-value pairs. Requires strict formatting (double quotes for keys).

Always set `Content-Type: application/json` in headers when sending!

★ 6. Status Codes (The Server's Reply)

2xx: Success

200 OK, 201 Created, 204 No Content

3xx: Redirection

301 Moved Permanently, 304 Not Modified (Cache hit)

4xx: Client Error

400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found

5xx: Server Error

500 Internal Error, 502 Bad Gateway, 503 Service Unavailable

Managing Users & Security

★ 9. AuthN vs AuthZ

Authentication (AuthN):

Identity. "Who are you?" (Login via Password, OAuth, Biometrics).

Authorization (AuthZ):

Permissions. "What can you do?" (RBAC - Role Based Access Control, Admin vs User).

★ 10. Cookies

Small text files stored in the browser. Sent automatically with every request to the issuing domain.

Crucial Security Flags:

- HttpOnly: Blocks JS access (prevents XSS stealing).
- Secure: Only transmits over HTTPS.
- SameSite=Strict/Lax: Mitigates CSRF attacks.

★ 11. Sessions (Stateful)

Server creates a session, stores data in DB/Redis, sends a generic `Session ID` via cookie.



Pros: Easy to revoke.

Cons: Harder to scale horizontally (needs shared DB/Redis).

★ 12. JWT (JSON Web Tokens - Stateless)

Server signs a payload and gives it to the client. The client sends it back (usually in `Authorization: Bearer` header). Server verifies signature without needing a database lookup.

JWT Structure: Header.Payload.Signature



⚠ JWT Security Guardrails

- ✗ **Never store sensitive data** (passwords, SSNs) in a JWT payload. It is Base64 encoded, anyone can decode it.
- ✗ **Avoid LocalStorage** for JWTs if possible. It's highly vulnerable to XSS. Best practice: Store in an HttpOnly cookie.

⇒ SCALE & SECURITY ⇒

Beyond the Basics

★ 13. CORS (Cross-Origin Resource Sharing)

A browser security feature. Prevents a malicious script on `evil.com` from making unauthorized requests to `your-bank.com`.

- **Same-Origin Policy:** By default, browsers block cross-origin requests.
- **Preflight Request:** Browser sends an `OPTIONS` request first to check if the server allows the origin.
- Server must respond with `Access-Control-Allow-Origin: \*` (or specific domain).

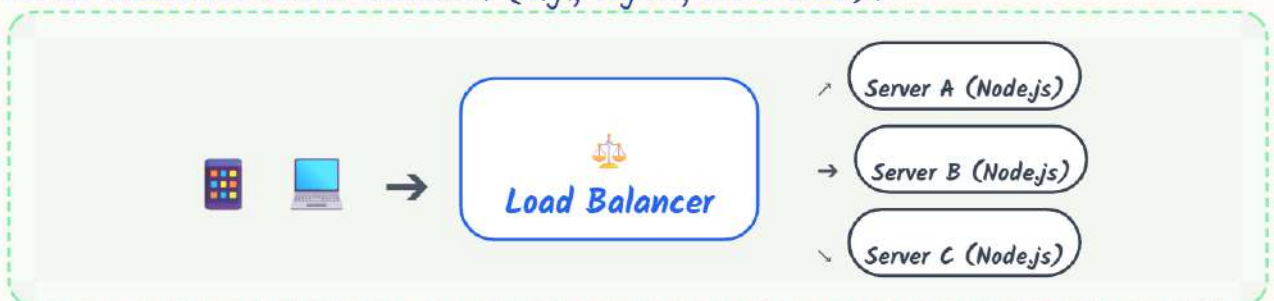
★ 14. WebSockets

Unlike HTTP (which is strictly request/response), WebSockets provide a **full-duplex, persistent connection** over a single TCP connection.

Use cases: Chat apps, live sports updates, real-time gaming.

★ 16. Load Balancing (Horizontal Scaling)

Distributing incoming network traffic across multiple servers to ensure no single server bears too much demand. (e.g., Nginx, AWS ALB).



★ 15. Caching Strategies

Storing copies of data to serve future requests faster. Crucial for performance.

- **Client-Side:** Browser Cache (Images, CSS). Managed via `Cache-Control` headers.
- **CDN (Content Delivery Network):** Caches static assets globally near the user (Cloudflare, AWS CloudFront).
- **Server-Side:** Redis/Memcached. Caching frequent DB query results.



Architectural Mindset

Building robust systems isn't just about knowing frameworks like React or Express. It's about deeply understanding **how data moves**, **how it's secured**, and **where the bottlenecks occur** at the protocol level.